

EA1_DIAGRAMA DE CLASES UML — SISTEMA DE BIBLIOTECA

PROGRAMACIÓN II: ORIENTADA A OBJETOS AVANZADA PREICA2602B010093

BORIS ALBERTO SALLEG

EMILSE CAVADIA TORDECILLA

IDENTIFICACION: CC 1033369434

emilse.cavadia@est.iudigital.edu.co

CRISTIAN CAMILO DEOSSA BOLIVAR

IDENTIFICACION: CC 1047994439

cristian.deossa@est.iudigital.edu.co

JOSE WILBER MURILLO MURILLO

IDENTIFICACION: CC 1010029229

wilber.murillo@est.iudigital.edu.co

KAROL LICETH PEREZ CASTAÑEDA

IDENTIFICACION: CC 101153607

karol.perez@est.iudigital.edu.co

NICOLE VALENTINA QUEVEDO FRANCO

IDENTIFICACION: CC 1025537804

nicole.quevedo@est.iudigital.edu.co

GRUPO 7

INSTITUCION UNIVERSITARIA DIGITAL DE ANTIOQUIA

25/08/2026

1. Introducción

El diagrama de clases UML es uno de los artefactos fundamentales dentro del modelado de software orientado a objetos, ya que permite una presentación clara y estructurada, sus atributos, métodos, relación y restricciones. Su importancia es debida a que funciona como un plano de diseño, facilitando la comunicación en el equipo de trabajo sirviendo de guía directa para la implementación de los códigos. En este trabajo se modela un sistema de gestión de biblioteca en base a la administración de autores, libros, usuarios y prestamos también viendo los estados de disponibilidad del material bibliográfico, el propósito de esta actividad es aplicar los conceptos del diseño de software mediante un trabajo colaborativo y un repositorio de GitHub el cual permite la integración de aportes, registrar los cambios y manteniendo un seguimiento ordenado.

2. Objetivos

2.1. Objetivo general

Diseñar un diagrama de clases UML para un sistema de biblioteca, aplicando relaciones entre clases, multiplicidades y principios de programación orientada a objetos de manera clara y organizada.

2.2 Objetivos específicos

- Abstracción: identificando las entidades esenciales del dominio, Autor, libro, usuario, préstamo, entendiendo sus características y sus comportamientos relevantes centrándose en lo necesario para el sistema.
- Encapsulamiento: establecer los niveles de accesos privados para los atributos de las clases, (ejemplo: idLibro, nombre, fechaNacimiento), con el fin de proteger los datos y

que garantice que únicamente se modificaran con los métodos definidos como prestar(), actualizarEstado() o registrar ()

- Herencia: Clase base Libro, clase derivada LibroDigital, heredando los atributos y métodos agregando características propias como, formato, tamañoArchivo métodos específicos como descargar() y leerEnLinea(), evitando que se repita y favoreciendo la reutilización
- Polimorfismo: Diseñando operaciones que pueden implementarse de una manera distinta teniendo en cuenta el tipo de objeto: por ejemplo, el comportamiento del préstamo consulta varía entre un libro físico y uno digital.
- Principios SOLID: Aplicando a cada principio en el diseño: la responsabilidad única, haciendo que cada clase cumpla con su función específica: ejemplo, EstadoDisponible nos muestra únicamente los estados de los libros, con el fin de lograr un sistema modular coherente y fácil de extender.

3. Descripción general del sistema

El sistema de biblioteca permite registrar libros y autores, administrar usuarios y controlar los préstamos. La clase Libro contiene la información general de una obra y su estado de disponibilidad. LibroDigital es una especialización de Libro para representar obras en formato digital. Autor permite identificar a los creadores de las obras y Usuario representa a las personas que realizan préstamos. Préstamo registra la operación de préstamo y devolución.

La enumeración EstadoDisponibilidad permite manejar estados como DISPONIBLE, PRESTADO, RESERVADO y EN_MANTENIMIENTO.

4. Clase del modelo

CLASE	ATRIBUTOS PRINCIPALES	MÉTODOS PRINCIPALES	FUNCIÓN
Autor	idAutor, nombre, nacionalidad, fechaNacimiento	escribirLibro(), obtenerInfo()	Representa al autor de una o varias obras.
Libro	idLibro, titulo, isbn, añoPublicacion, estado	prestar(), actualizarEstado(), obtenerInfo()	Representa un libro y su estado dentro de la biblioteca.
LibroDigital	formato, tamañoArchivo, urlDescarga, protegido	descargar(), leerEnLinea()	Especialización de Libro para obras digitales.
Préstamo	idPréstamo, fechaPréstamo, fechaDevolucion, fechaDevolucionReal, estado	registrar(), devolver(), calcularDiasAtraso()	Registra el préstamo y la devolución.
Usuario	idUsuario, nombre, documento, tipoUsuario	registrar(), obtenerInfo()	Representa a la persona que realiza préstamos.

5. Diagrama de clase UML

El siguiente diagrama corresponde al modelo propuesto generado en PlantUML a partir del código incluido en la sección siguiente.

DIAGRAMA DE CLASES UML - SISTEMA DE BIBLIOTECA

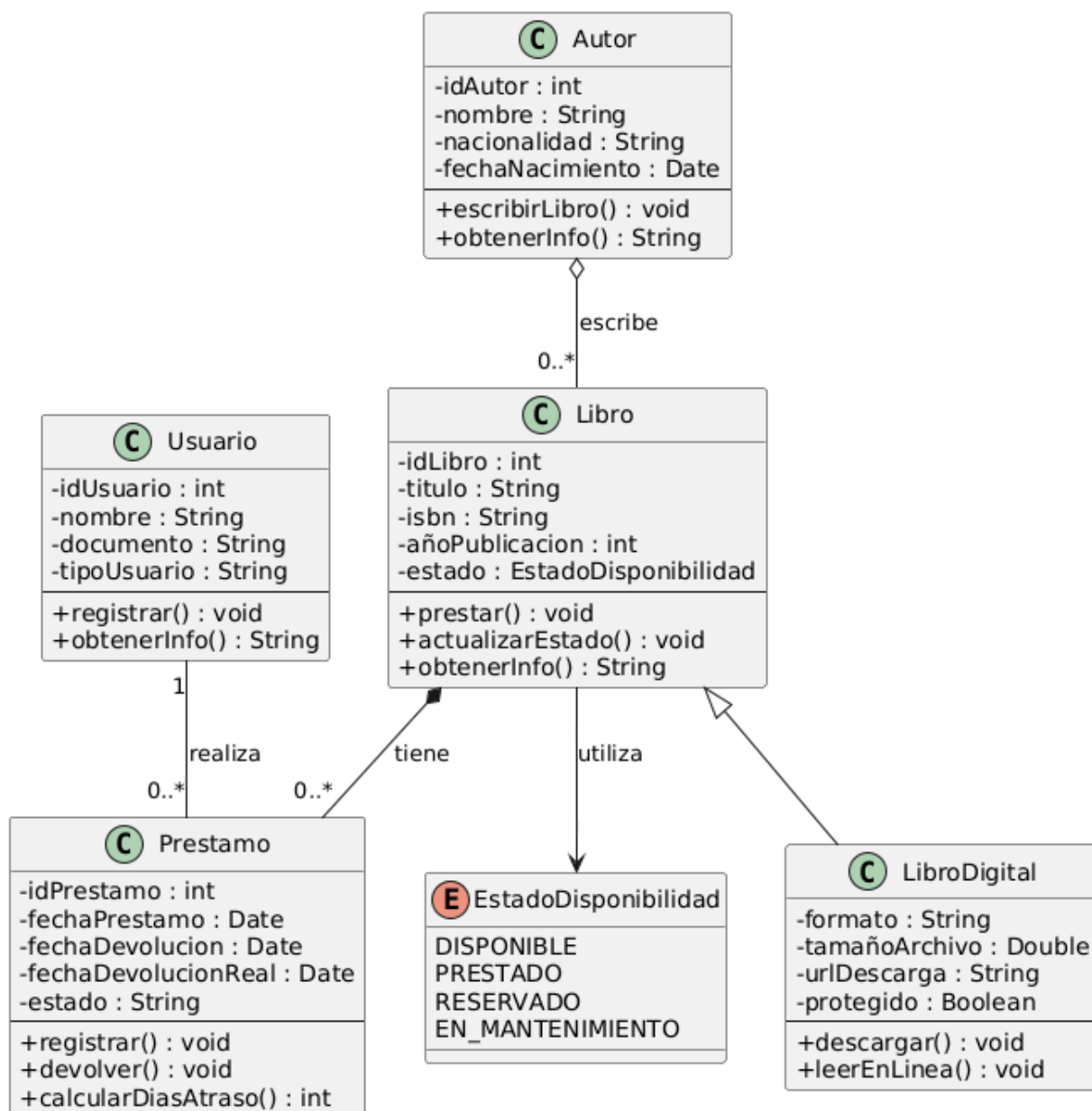


Figura 1. Diagrama de clases UML del sistema de biblioteca. Elaboración propia.

6. Código PlantUML

El siguiente código permite reconstruir el diagrama en PlantUML. Se mantienen las clases, atributos, métodos, enumeración, relaciones y multiplicidades del modelo.

```
@startuml

title DIAGRAMA DE CLASES UML - SISTEMA DE BIBLIOTECA

skinparam classAttributeIconSize 0
skinparam shadowing false

class Autor {
    - idAutor : int
    - nombre : String
    - nacionalidad : String
    - fechaNacimiento : Date
    --
    + escribirLibro() : void
    + obtenerInfo() : String
}

class Libro {
    - idLibro : int
    - titulo : String
    - isbn : String
    - añoPublicacion : int
    - estado : EstadoDisponibilidad
    --
    + prestar() : void
    + actualizarEstado() : void
    + obtenerInfo() : String
}
```

```
class LibroDigital {  
    - formato : String  
    - tamañoArchivo : Double  
    - urlDescarga : String  
    - protegido : Boolean  
    --  
    + descargar() : void  
    + leerEnLinea() : void  
}  
  
class Prestamo {  
    - idPrestamo : int  
    - fechaPrestamo : Date  
    - fechaDevolucion : Date  
    - fechaDevolucionReal : Date  
    - estado : String  
    --  
    + registrar() : void  
    + devolver() : void  
    + calcularDiasAtraso() : int  
}  
  
class Usuario {  
    - idUsuario : int  
    - nombre : String  
    - documento : String  
    - tipoUsuario : String  
    --  
    + registrar() : void  
    + obtenerInfo() : String  
}  
  
enum EstadoDisponibilidad {  
    DISPONIBLE  
    PRESTADO
```

```
RESERVADO  
EN_MANTENIMIENTO  
}  
  
Autor o-- "0..*" Libro : escribe  
Libro *-- "0..*" Prestamo : tiene  
Usuario "1" -- "0..*" Prestamo : realiza  
Libro <|-- LibroDigital  
Libro --> EstadoDisponibilidad : utiliza  
  
@enduml
```

7. Relaciones UML

7.1. Asociación: Usuario – Préstamo

La asociación indica que un Usuario puede realizar cero o muchos préstamos, mientras que cada Préstamo corresponde a un solo Usuario. En PlantUML se representa como: Usuario "1" -- "0..*" Prestamo : realiza.

7.2. Asociación / agregación: Autor – Libro

Autor y Libro pueden existir de manera independiente. El modelo muestra la relación de asociación y, como parte de la actividad, la agregación se representa mediante un rombo vacío. Un autor puede estar relacionado con cero o muchos libros.

7.3. Composición: Libro – Préstamo

La composición se representa con un rombo lleno. En el modelo se utiliza para expresar una relación fuerte entre Libro y los registros de Préstamo. Un libro puede tener cero o muchos préstamos registrados.

7.4. Herencia: Libro - LibroDigital

LibroDigital hereda de Libro. La herencia permite reutilizar los atributos y comportamientos generales de Libro y añadir características propias del formato digital, como formato, tamaño del archivo y URL de descarga.

7.5. Multiplicidades

Las multiplicidades indican la cantidad de objetos que pueden participar en una relación. 1 significa exactamente uno; 0..* significa cero o muchos; 1..* significa uno o muchos y 0..1 significa cero o uno.

8. Abstracción, encapsulamiento y polimorfismo

8.1. Abstracción

La abstracción consiste en representar solamente la información importante para el sistema. Por ejemplo, Libro contiene los datos y operaciones que se necesitan para gestionar una obra en la biblioteca, sin representar detalles que no son necesarios para el modelo.

8.2. Encapsulamiento

Los atributos se plantean como privados y los métodos como públicos. Esto ayuda a proteger la información interna de cada objeto y evita que cualquier parte del programa modifique directamente los datos.

8.3. Polimorfismo

El polimorfismo permite trabajar con diferentes tipos de libros a través de una estructura común. LibroDigital puede utilizar los comportamientos generales de Libro y, al mismo tiempo, incorporar comportamientos propios como descargar o leer en línea.

9. Principios SOLID

S – Responsabilidad única: Cada clase tiene una responsabilidad principal: Libro gestiona información del libro, Autor representa autores, Usuario representa usuarios y Préstamo gestiona préstamos.

O – Abierto/cerrado: El modelo puede ampliarse mediante nuevas especializaciones de Libro sin tener que modificar toda la estructura.

L – Sustitución de Liskov: Un objeto LibroDigital debe poder utilizarse donde se espera un Libro sin romper el funcionamiento general.

I – Segregación de interfaces: Si se implementan interfaces posteriormente, se recomienda que cada clase dependa solo de las operaciones que realmente necesita.

D – Inversión de dependencias: En una implementación real, los componentes principales pueden depender de abstracciones para facilitar cambios, pruebas y mantenimiento.

10. CONCLUSIÓN

¿Qué se aprendió?

- Mediante el desarrollo del diagrama se permite comprender y aplicar de forma práctica lo que son los fundamentos del diseño orientado a objetos, se ve reflejado el uso de la herencia en la relación que hay entre Libro y LibroDigital, donde se está incorporando las características propias del formato digital, reduciendo que el código se duplique. El encapsulamiento se evidencia el control de acceso a los atributos de cada entidad ayudando en la protección de la información interna y garantizando su modificación mediante la operación controlada. El polimorfismo se determina en la capacidad de la gestión de manera diferenciada en el préstamo de y consultas de libros físicos y digitales teniendo una adaptación del comportamiento en base al tipo de objetos. Los principios de SOLID guiaron las decisiones como la creación de la clase EstadoDisponible para centrarse en la gestión de estados cumpliendo así con su única responsabilidad y la estructura de clases de manera que el sistema tenga la capacidad de ampliarse sin alterar la base, teniendo así un dominio sólido de los conceptos del diseño.

¿En qué aporta su desempeño profesional?

- El modelo UML y el trabajo colaborativo tienen como fin competencias esenciales para el ejercicio en el desarrollo de software, el uso de GitHub y Git nos permiten fomentar la organización, la trazabilidad de algún cambio y la capacidad de trabajar de forma sincrónica y asincrónica con otros desarrolladores, la elaboración del diagrama nos favorece la documentación técnica clara u estandarizada, teniendo la facilidad de poder comprender el sistema a futuros integrantes reduciendo los tiempos de mantenibilidad del código ya que el

diseño bien estructurado simplifica las correcciones y mejoras. Estas herramienta y habilidades promueven a buenas prácticas para un diseño, pensamientos estructurados y el trabajo en equipo, elementos fundamentales para construir soluciones tecnológicas, escalables y que sean sostenibles a largo plazo.

11. ENLACES DEL DIAGRAMA, ENLACE REPOSITORIO GitHub Y VIDEO

- **ENLACE DIAGRAMA:**

www.plantuml.com/plantuml/png/dPJDRjim3CVIVWeXfws3Y-vQ34FTsXWCv2E8qrq7HcPIOh8KI5G6TDj3xHdsObDafNMJeDYMIqp-IO8_KTID0x3iEvLbJAnGb7NnQL5C2b5MucvSD5KZxYTZaOkcRfPLxx-jRyVrR5dT5LdclfBUWeLEI0NE5SoMLfwnbaOtz8ZYQX3Y7g0rtqXln1gKmooBEQBmRApudedmomMrlNqjI7FoQTEjB0PNmpQa7xmWoMXGr49xhArHFi0q17I4casGIs3inJp-lHFeP6YMx9XMrhmv6s9sXjeacXMZHBhlTI-bwZyFJSUaGTEzFMoQYRqonusHM-bZ7_p-PUP-fKX6eXTLq36q-ofL_2Z9ROqENKVgbpnRkm-snoGWsOEYHx2nnF-HbhGX1lK4l3Qs0pwXO-ZsC8ML3xIBfsu25oRLMrMYas0tS9oujOPnGv7qrXY5e5_2jIdn14yXsagFIICyIq_Dp-EPTCD99TTmMEc-79JXRKbIYJkZV9pEQ-82GQM0u_aDSGzGDY0xjgTGxRxY3a_y4fJq2cn9u8gGwCB84iSJyhtpO6b8d3n_iLAjaRv3VMQmj3Mfp3-ol7A_KFlkxBrElPTrCvzDmsjJHNE-g9fbKSwY4RwhnUU3LKs_J8hfifhMaxgQBcVxwlpOl9SZAukBoz68gresdlCUIIYEM0mVoRKcl8H-z7ucoOnRXPgTI77o5ek2x9d-SVpsDU2yyK2sGtgDlZE_m7

- **ENLACE REPOSITORIO GitHub:** <https://github.com/Karol-perez-castaneda/Evidencia1-Diagrama-de-clase-UML.git>

- **ENLACE VIDEO:**

<https://drive.google.com/file/d/16QTO7JstaKnW3YINzWOJpNELQYrKvnMW/view?usp=drivesdk>

12. REFERENCIAS BIBLIOGRÁFICAS

- IBM. (2025). ¿Qué es la normalización de bases de datos? IBM Think.
<https://www.ibm.com/es-es/think/topics/database-normalization>
- IBM. (s. f.). Normalization. IBM Documentation.
<https://www.ibm.com/docs/en/tpfdf/1.1.3?topic=database-normalization>
- Oracle. (s. f.). About normalization. Oracle Database Data Warehousing Guide.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/dwhsg/data-warehouse-logical-design.html>
- Otaduy, I. (s. f.). Ingeniería del software: Bloque II: UML (1) [Archivo PDF]. Universidad del País Vasco / Euskal Herriko Unibertsitatea.
<http://www.vc.ehu.es/jiwotvim/IngenieriaSoftware/Teoria/BloqueII/UML-1.pdf>
- Miro. (s. f.). ¿Qué es un diagrama UML? <https://miro.com/es/diagrama/que-es-diagrama-uml/>